

Tripistry: PA5_COS221

Group 35: The Spice Girls

https://github.com/UncleShak/COS221_PA5

Members:

- Shakir Alexander
- Frank Yambayamba
- Leul Tamiru
- Aeron Vaidyan
- Thakasa Ngcobo

Task 1:

Research on Online Travel Planning and Price Comparison Platforms for a Multi-Product Travel Package System

Arranging a vacation period nowadays involves moving through many different digital locations. Travelers open one digital location for flights and another for sleeping places while looking for good dining establishments nearby or reading a digital diary for ideas about attractions. Because searching through many digital locations takes time, the process brings frustration to the traveller. Although travel platforms exist to bring these details together, many big travel platforms do not permit a person to compare complete vacation bundles across different providers. This document examines how current travel platforms operate and looks at the types of agencies and products that exist in the market. Many experts claim that the logic of ratings and how things are grouped needs study to see which features like group voyages help the traveller. The Tripistry system is built specifically around an agency driven model that focuses on multiple products.

Different structures exist for the way travel platforms are organized. Online Travel Agencies like Booking.com and Expedia collect flights, rooms, and vehicle hire services so they can take a payment from every booking. Prices are compared across multiple digital locations by metasearch engines like Skyscanner and Kayak even though these engines do not manage the booking process themselves. Because some travel agencies create ready-made vacation bundles, package holiday platforms like Jet2Holidays, TUI, and Tripmasters sell these combined services. These bundles usually include flights and a sleeping place while sometimes adding transport or activities to the mix. Tripistry follows this agency driven package model. Agencies organize curated bundles that can gather geographic locations, flights, sleeping places, attractions, and dining establishments into a single choice. Because travellers look at these bundles across different agencies, the comparison feature is very important. It has been observed that users prefer comparing fully planned voyages side by side instead of looking at separate flight prices.

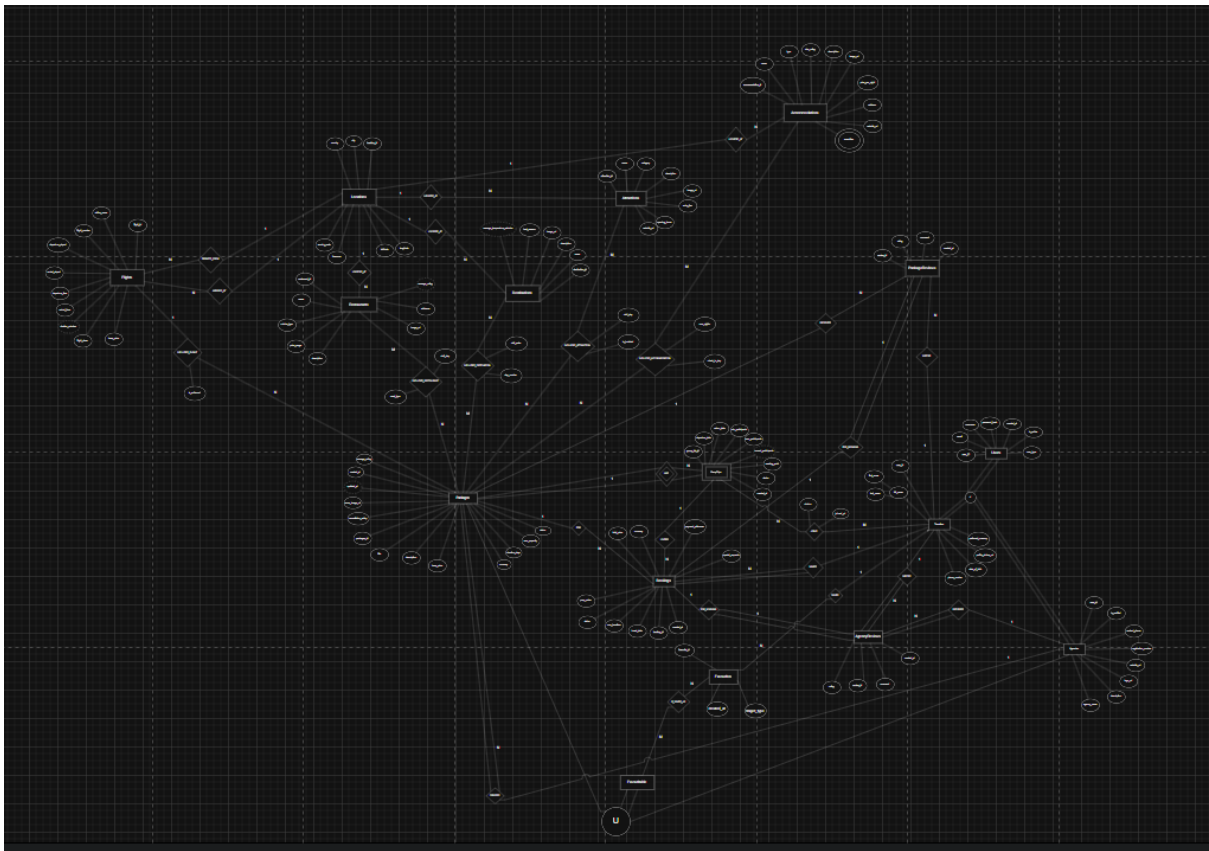
A mix of star ratings and user feedback is used by most travel platforms to help people choose. Sleeping places are usually given a rating between one and five stars because of their standards and services. Specific amenities like wireless internet access, a pool, parking, or a morning meal are selected by users when they filter results on accommodation sites. Feedback from other travellers carries a lot of weight when a person decides what to book. Many people believe that user-generated feedback is the most important factor because platforms like TripAdvisor built their whole business on people rating dining establishments and attractions. Online reviews are shown by studies to have a strong effect on how people behave when booking. Because sentiment analysis of feedback text is possible, the technology can be used to rank agencies automatically. This feedback system applies specifically to travel agencies within Tripistry. Ratings and feedback are left by travellers for the agencies they used which helps future users know if an agency is trustworthy. Because the platform provides filtering tools, bundles can be sorted by geographic location, price, and the length of stay.

Modern digital travel platforms provide extra tools that make the arrangement steps less complicated instead of just performing reservation services. Arranging group travel represents one such helpful tool. Various computer applications permit several individuals to cooperate on one travel schedule where individuals can choose specific choices and perform the group reservation together. It has been observed that tailored recommendations driven by computer intelligence are becoming more popular in the industry today. Digital travel platforms study the history of what a person searched or reserved in the past so that the digital travel platforms suggest specific locations or bundles which that person might enjoy. The system of Tripistry contains various concepts which were mentioned before. Every group trip is integrated into the computer database as a foundational component of the system. While agencies develop group travel offers that are connected to a package, specific voyagers are allowed to register for those group travel offers. The digital travel platform also facilitates the gathering of several items into one single package. This package is not restricted to a flight and a guest room in a lodging house because this package also includes sightseeing locations and food establishments. Through the inclusion of these items, a more thorough arrangement is given to voyagers within one reservation. Experts claim that the feedback method resolves the issues of honesty and clarity which were highlighted in the project brief. The previous comments of other people are viewed by voyagers so that voyagers feel safe before spending financial resources on an agency.

References (IEEE):

- [1] J. K. Kim, W. C. Kwon, I. Y. Choi, H. Heo, and H. S. Moon, "A Group Travel Recommender System Based on Group Approximate Constraint Satisfaction," *IEEE Access*, vol. 12, pp. 96113-96125, 2024.
- [2] S. Staab, H. Werthner, F. Ricci, A. Zipf, U. Gretzel, D. R. Fesenmaier, C. Paris, and C. Knoblock, "Intelligent Systems for Tourism," *IEEE Intelligent Systems*, vol. 17, no. 6, pp. 53-66, 2002.
- [3] L. Peng and L. Zhang, "Study on Business Models Transformation of Online Travel Services Industry," in *2010 Int. Conf. on E-Business and E-Government*, 2010, pp. 3042-3045.
- [4] Y. Li, Y. Tong, F. Ye, Y. J. Chen, and X. Zhao, "Sales Channel Choice in the Hotel Industry: The Impact of Online Reviews," *IEEE Trans. on Engineering Management*, 2022.

Task 2:



Task 3:

Step 1:

Diagram illustrating the database schema for a travel agency, showing tables and their relationships.

LOCATIONS

location_id	city	country	country_code	timezone	latitude	longitude
-------------	------	---------	--------------	----------	----------	-----------

DESTINATIONS

destination_id	name	description	image_url	best_season	average_temperature_celsius
----------------	------	-------------	-----------	-------------	-----------------------------

FLIGHTS

flight_id	airline_name	flight_number	departure_airport	arrival_airport	departure_time	arrival_time	duration_minutes	flight_class	base_price	departure_location_id	arrival_location_id
-----------	--------------	---------------	-------------------	-----------------	----------------	--------------	------------------	--------------	------------	-----------------------	---------------------

ACCOMMODATIONS

accommodation_id	name	type	star_rating	description	image_url	price_per_night	address	website_url
------------------	------	------	-------------	-------------	-----------	-----------------	---------	-------------

ATTRACTIONS

attraction_id	name	category	description	image_url	entry_fee	opening_hours	website_url
---------------	------	----------	-------------	-----------	-----------	---------------	-------------

RESTAURANTS

restaurant_id	name	cuisine_type	price_range	description	image_url	address	average_rating
---------------	------	--------------	-------------	-------------	-----------	---------	----------------

PACKAGES

package_id	title	description	base_price	currency	duration_days	max_capacity	status	created_at	updated_at	cover_image_url	cancellation_policy
------------	-------	-------------	------------	----------	---------------	--------------	--------	------------	------------	-----------------	---------------------

BOOKINGS

booking_id	travel_date	num_travelers	total_price	currency	status	payment_reference	special_requests	created_at
------------	-------------	---------------	-------------	----------	--------	-------------------	------------------	------------

AGENCYREVIEWS

review_id	rating	comment	created_at
-----------	--------	---------	------------

PACKAGEREVIEWS

review_id	rating	comment	created_at
-----------	--------	---------	------------

FAVOURITES

favorite_id	created_at
-------------	------------

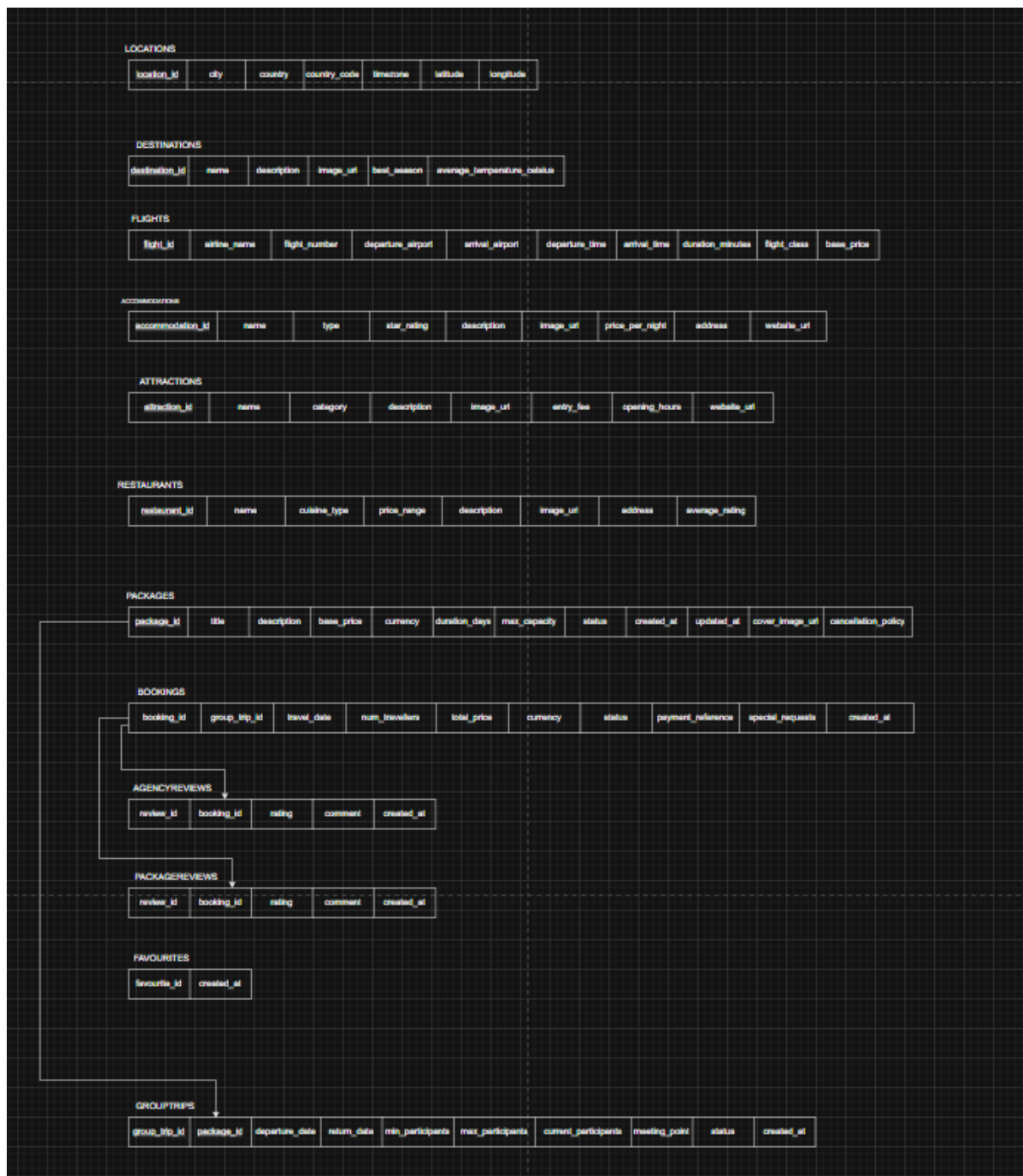
Relationships:

- LOCATIONS (location_id) to DESTINATIONS (destination_id)
- LOCATIONS (location_id) to FLIGHTS (departure_location_id)
- LOCATIONS (location_id) to FLIGHTS (arrival_location_id)
- FLIGHTS (flight_id) to ACCOMMODATIONS (accommodation_id)
- FLIGHTS (flight_id) to ATTRACTIONS (attraction_id)
- FLIGHTS (flight_id) to RESTAURANTS (restaurant_id)
- FLIGHTS (flight_id) to PACKAGES (package_id)
- FLIGHTS (flight_id) to BOOKINGS (booking_id)
- FLIGHTS (flight_id) to AGENCYREVIEWS (review_id)
- FLIGHTS (flight_id) to PACKAGEREVIEWS (review_id)
- FLIGHTS (flight_id) to FAVOURITES (favorite_id)

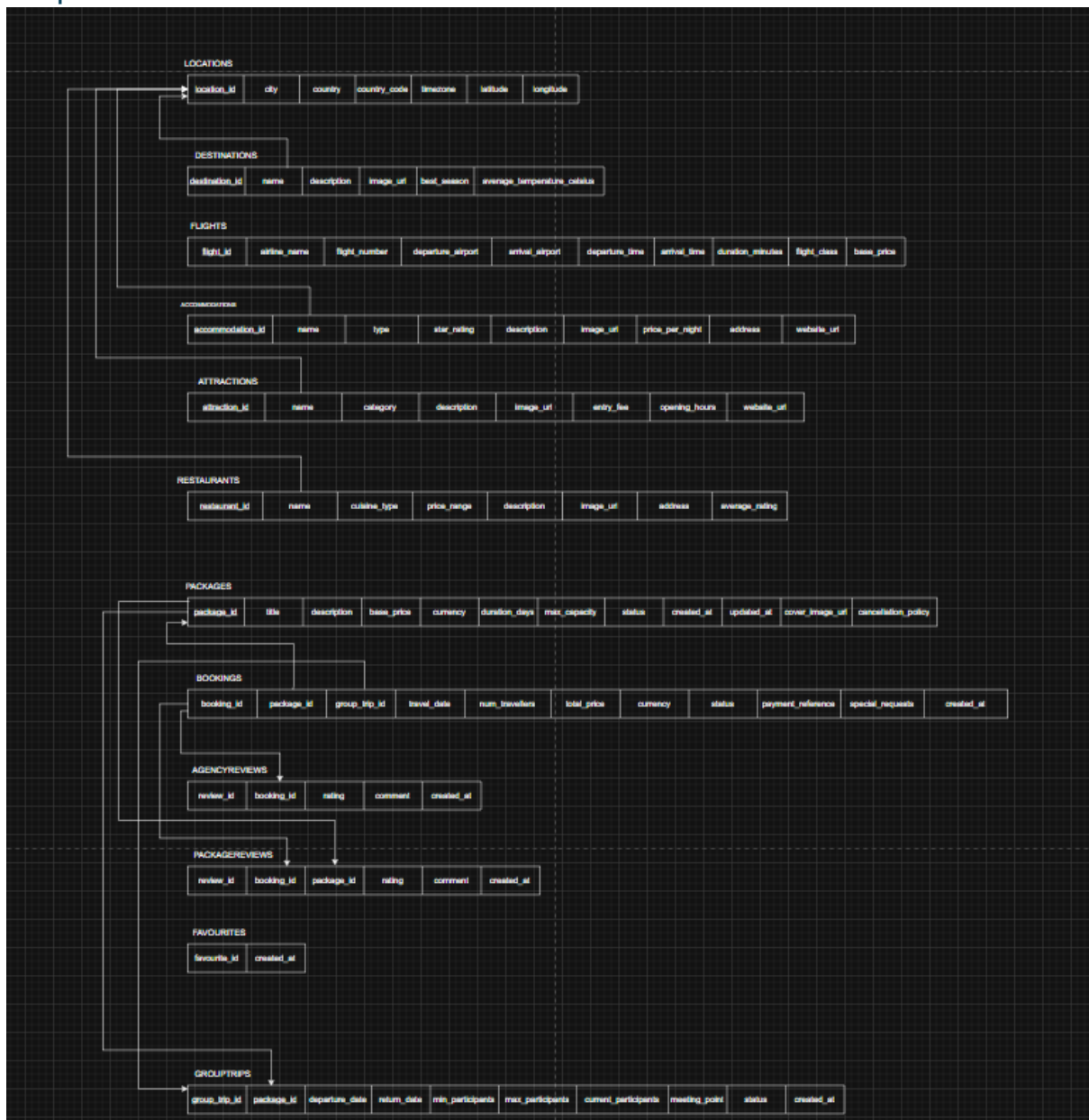
Step 2:



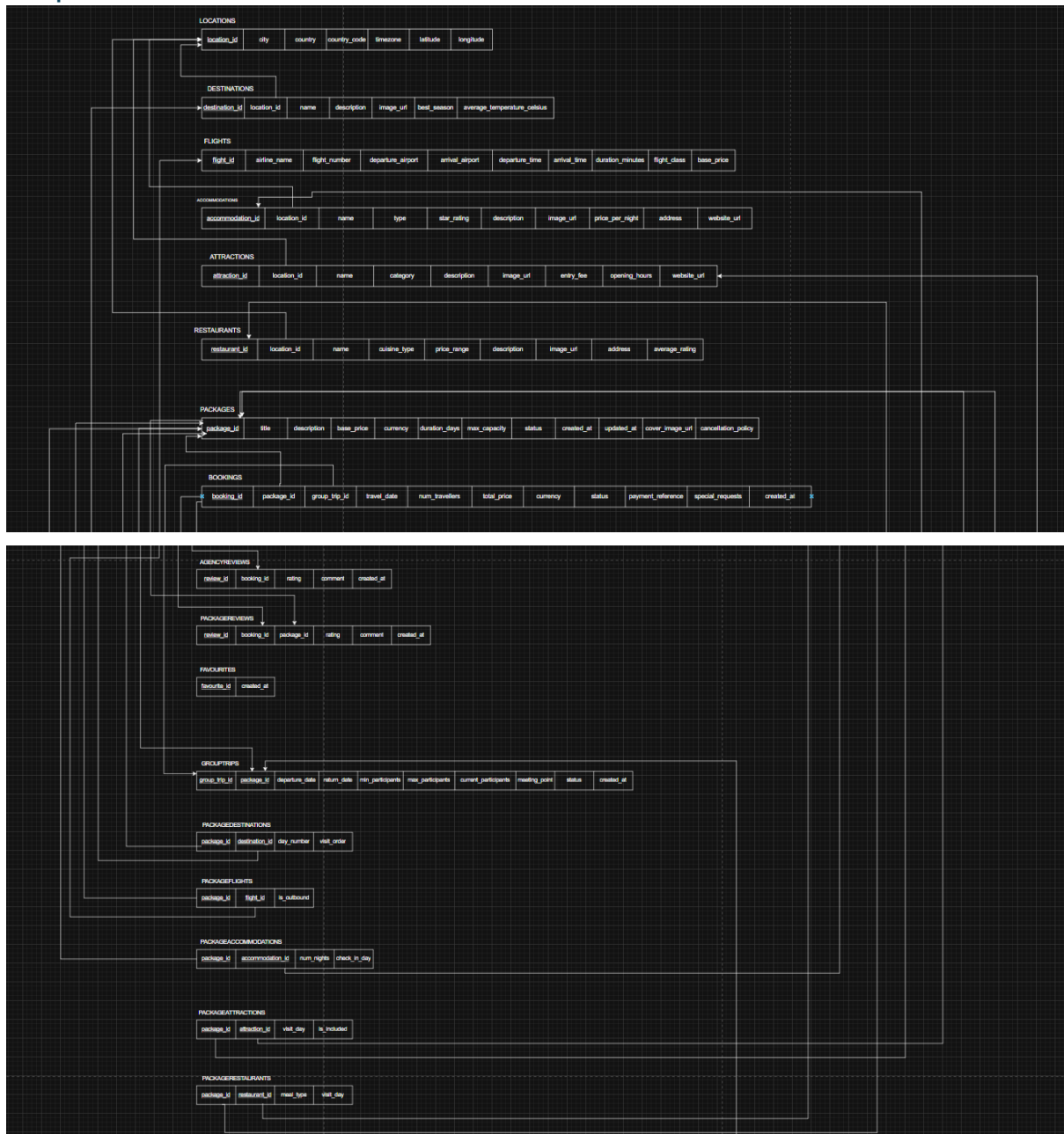
Step 3:



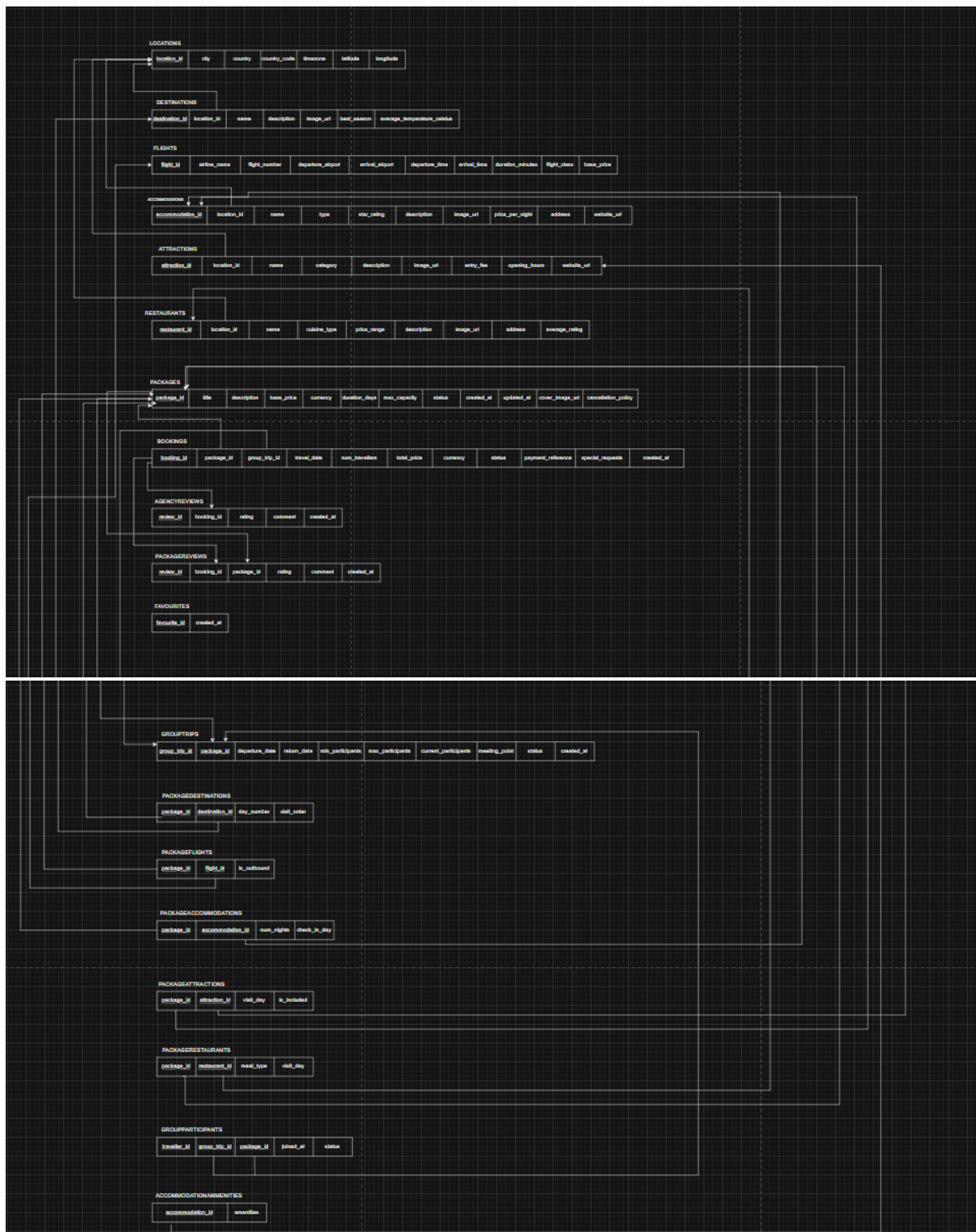
Step 4:



Step 5:



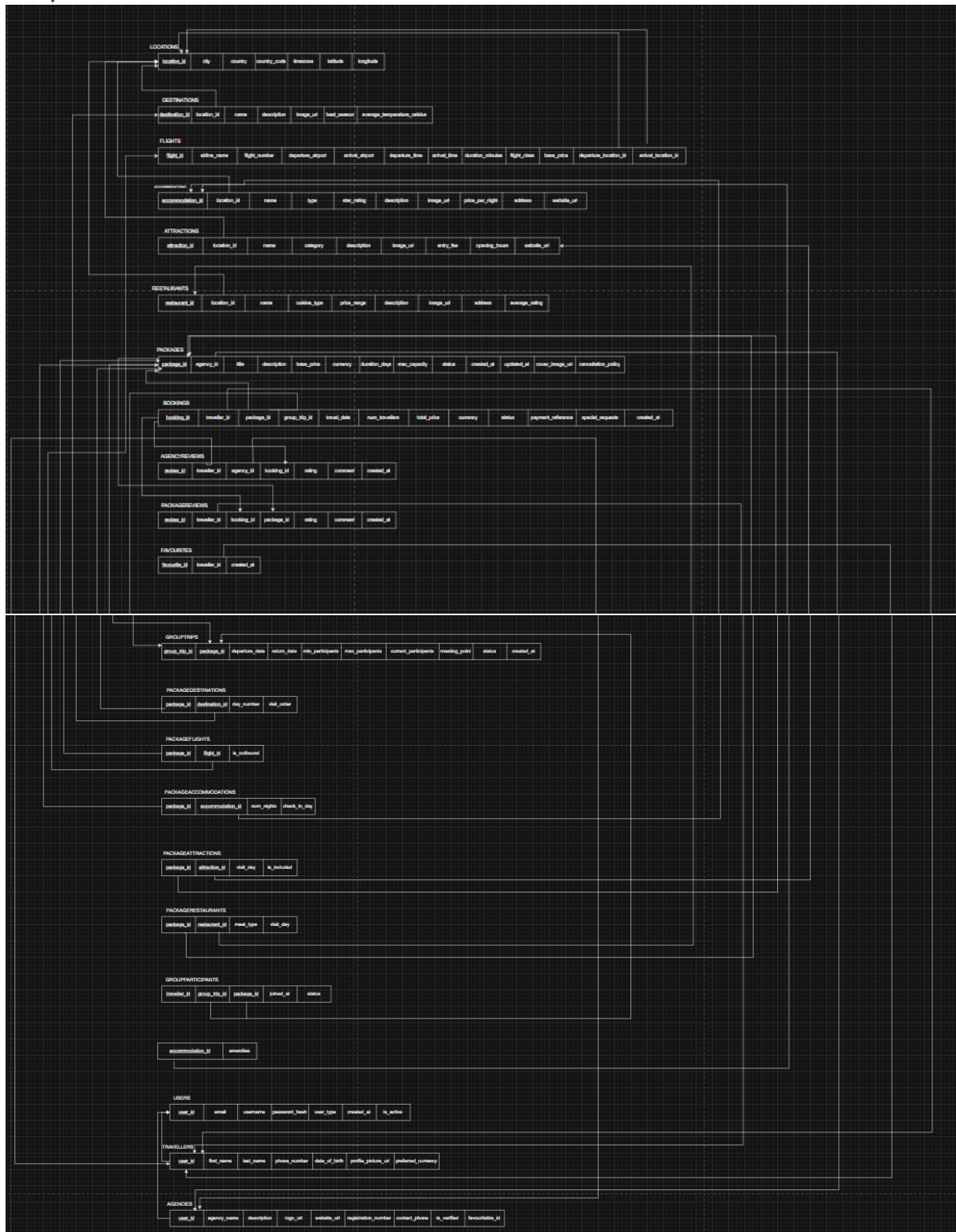
Step 6:



Step 7:



Step 8:



Step 9:

1



Task 4:

Here are all the SQL commands:

```
CREATE DATABASE IF NOT EXISTS tripistry;

USE tripistry;

-- 1. Base Tables

CREATE TABLE users (user_id INT AUTO_INCREMENT PRIMARY KEY, email VARCHAR(255) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, user_type ENUM('traveller', 'agency') NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP) ENGINE=InnoDB;

CREATE TABLE locations (location_id INT AUTO_INCREMENT PRIMARY KEY, city VARCHAR(150) NOT NULL, country VARCHAR(100) NOT NULL, country_code CHAR(2) NOT NULL, timezone VARCHAR(100), latitude DECIMAL(10,7), longitude DECIMAL(10,7)) ENGINE=InnoDB;

CREATE TABLE favouritable (favouritable_id INT AUTO_INCREMENT PRIMARY KEY, target_type ENUM('agency', 'package') NOT NULL) ENGINE=InnoDB;

-- 2. User Profiles

CREATE TABLE travellers (user_id INT PRIMARY KEY, first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL, phone_number VARCHAR(20), date_of_birth DATE, preferred_currency CHAR(3) DEFAULT 'ZAR', CONSTRAINT fk_travellers_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE) ENGINE=InnoDB;

CREATE TABLE agencies (user_id INT PRIMARY KEY, agency_name VARCHAR(255) UNIQUE NOT NULL, is_verified TINYINT(1) DEFAULT 0, favouritable_id INT UNIQUE, CONSTRAINT fk_agencies_user FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE, CONSTRAINT fk_agencies_fav FOREIGN KEY (favouritable_id) REFERENCES favouritable(favouritable_id)) ENGINE=InnoDB;

-- 3. Inventory Tables

CREATE TABLE accommodations (accommodation_id INT AUTO_INCREMENT PRIMARY KEY, location_id INT NOT NULL, name VARCHAR(255) NOT NULL, type ENUM('hotel', 'guesthouse', 'resort', 'hostel', 'apartment', 'villa') NOT NULL, star_rating TINYINT, price_per_night DECIMAL(10,2) NOT NULL, CONSTRAINT fk_acc_loc FOREIGN KEY (location_id) REFERENCES locations(location_id)) ENGINE=InnoDB;

CREATE TABLE destinations (destination_id INT AUTO_INCREMENT PRIMARY KEY, location_id INT NOT NULL, name VARCHAR(255) NOT NULL, best_season ENUM('spring', 'summer', 'autumn', 'winter', 'year-round'), CONSTRAINT fk_dest_loc FOREIGN KEY (location_id) REFERENCES locations(location_id)) ENGINE=InnoDB;

CREATE TABLE flights (flight_id INT AUTO_INCREMENT PRIMARY KEY, airline_name VARCHAR(150) NOT NULL, departure_airport VARCHAR(10) NOT NULL, arrival_airport VARCHAR(10) NOT NULL, base_price DECIMAL(10,2) NOT NULL) ENGINE=InnoDB;

CREATE TABLE attractions (attraction_id INT AUTO_INCREMENT PRIMARY KEY, location_id INT NOT NULL, name VARCHAR(255) NOT NULL, category ENUM('museum', 'park', 'landmark', 'beach', 'historical', 'entertainment', 'other'), CONSTRAINT fk_attr_loc FOREIGN KEY (location_id) REFERENCES locations(location_id)) ENGINE=InnoDB;

CREATE TABLE restaurants (restaurant_id INT AUTO_INCREMENT PRIMARY KEY, location_id INT NOT NULL, name VARCHAR(255) NOT NULL, price_range ENUM('budget', 'mid-range', 'fine-dining'), CONSTRAINT fk_rest_loc FOREIGN KEY (location_id) REFERENCES locations(location_id)) ENGINE=InnoDB;

-- 4. Packages & Bookings

CREATE TABLE packages (package_id INT AUTO_INCREMENT PRIMARY KEY, agency_id INT NOT NULL, favouritable_id INT UNIQUE, title VARCHAR(255) NOT NULL, base_price DECIMAL(10,2) NOT NULL, duration_days INT NOT NULL, status ENUM('draft', 'active', 'archived') DEFAULT 'draft', CONSTRAINT fk_pkg_agency FOREIGN KEY (agency_id) REFERENCES agencies(user_id), CONSTRAINT fk_pkg_fav FOREIGN KEY (favouritable_id) REFERENCES favouritable(favouritable_id)) ENGINE=InnoDB;
```



```
CREATE TABLE bookings (booking_id INT AUTO_INCREMENT PRIMARY KEY, traveller_id INT NOT NULL,
package_id INT NOT NULL, status ENUM('pending','confirmed','cancelled','completed') DEFAULT 'pending',
CONSTRAINT fk_book_trav FOREIGN KEY (traveller_id) REFERENCES travellers(user_id), CONSTRAINT
fk_book_pkg FOREIGN KEY (package_id) REFERENCES packages(package_id)) ENGINE=InnoDB;
```

-- 5. Linking Tables (Associative Entities)

```
CREATE TABLE packageaccommodations (package_id INT, accommodation_id INT, PRIMARY
KEY(package_id, accommodation_id), CONSTRAINT fk_pa_pkg FOREIGN KEY (package_id) REFERENCES
packages(package_id) ON DELETE CASCADE, CONSTRAINT fk_pa_acc FOREIGN KEY (accommodation_id)
REFERENCES accommodations(accommodation_id)) ENGINE=InnoDB;
```

```
CREATE TABLE packageattractions (package_id INT, attraction_id INT, PRIMARY KEY(package_id,
attraction_id), CONSTRAINT fk_pat_pkg FOREIGN KEY (package_id) REFERENCES packages(package_id)
ON DELETE CASCADE, CONSTRAINT fk_pat_attr FOREIGN KEY (attraction_id) REFERENCES
attractions(attraction_id)) ENGINE=InnoDB;
```

```
CREATE TABLE packagedestinations (package_id INT, destination_id INT, PRIMARY KEY(package_id,
destination_id), CONSTRAINT fk_pd_pkg FOREIGN KEY (package_id) REFERENCES packages(package_id)
ON DELETE CASCADE, CONSTRAINT fk_pd_dest FOREIGN KEY (destination_id) REFERENCES
destinations(destination_id)) ENGINE=InnoDB;
```

```
CREATE TABLE packageflights (package_id INT, flight_id INT, PRIMARY KEY(package_id, flight_id),
CONSTRAINT fk_pf_pkg FOREIGN KEY (package_id) REFERENCES packages(package_id) ON DELETE
CASCADE, CONSTRAINT fk_pf_fit FOREIGN KEY (flight_id) REFERENCES flights(flight_id)) ENGINE=InnoDB;
```

```
CREATE TABLE packagerestaurants (package_id INT, restaurant_id INT, PRIMARY KEY(package_id,
restaurant_id), CONSTRAINT fk_pr_pkg FOREIGN KEY (package_id) REFERENCES packages(package_id)
ON DELETE CASCADE, CONSTRAINT fk_pr_rest FOREIGN KEY (restaurant_id) REFERENCES
restaurants(restaurant_id)) ENGINE=InnoDB;
```

```
CREATE TABLE accommodationamenities (accommodation_id INT, amenity VARCHAR(100), PRIMARY
KEY(accommodation_id, amenity), CONSTRAINT fk_aa_acc FOREIGN KEY (accommodation_id)
REFERENCES accommodations(accommodation_id) ON DELETE CASCADE) ENGINE=InnoDB;
```

-- 6. Social & Reviews

```
CREATE TABLE agencyreviews (review_id INT AUTO_INCREMENT PRIMARY KEY, traveller_id INT NOT
NULL, agency_id INT NOT NULL, booking_id INT UNIQUE, rating TINYINT CHECK (rating BETWEEN 1 AND 5),
CONSTRAINT fk_ar_trav FOREIGN KEY (traveller_id) REFERENCES travellers(user_id), CONSTRAINT
fk_ar_ag FOREIGN KEY (agency_id) REFERENCES agencies(user_id), CONSTRAINT fk_ar_bk FOREIGN KEY
(booking_id) REFERENCES bookings(booking_id)) ENGINE=InnoDB;
```

```
CREATE TABLE packagereviews (review_id INT AUTO_INCREMENT PRIMARY KEY, traveller_id INT NOT
NULL, package_id INT NOT NULL, booking_id INT UNIQUE, rating TINYINT CHECK (rating BETWEEN 1 AND
5), CONSTRAINT fk_pr_trav FOREIGN KEY (traveller_id) REFERENCES travellers(user_id), CONSTRAINT
fk_pr_pkg FOREIGN KEY (package_id) REFERENCES packages(package_id), CONSTRAINT fk_pr_bk
FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)) ENGINE=InnoDB;
```

```
CREATE TABLE favourites (favourite_id INT AUTO_INCREMENT PRIMARY KEY, traveller_id INT NOT NULL,
favouritable_id INT NOT NULL, CONSTRAINT fk_fav_trav FOREIGN KEY (traveller_id) REFERENCES
travellers(user_id), CONSTRAINT fk_fav_fav FOREIGN KEY (favouritable_id) REFERENCES
favouritable(favouritable_id)) ENGINE=InnoDB;
```

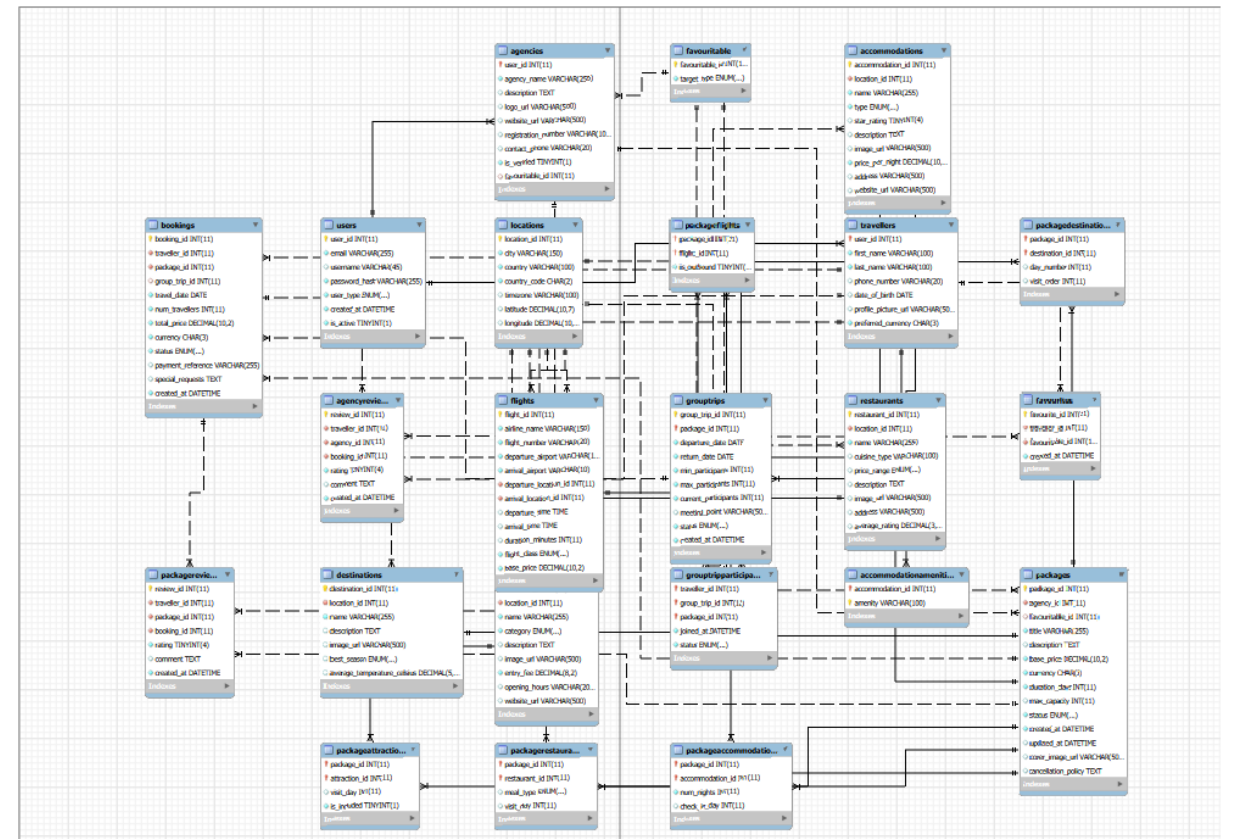
-- 7. Group Trips

```
CREATE TABLE grouptrips (group_trip_id INT AUTO_INCREMENT, package_id INT, departure_date DATE NOT
NULL, max_participants INT NOT NULL, status ENUM('open','full','departed','cancelled') DEFAULT 'open',
PRIMARY KEY (group_trip_id, package_id), CONSTRAINT fk_gt_pkg FOREIGN KEY (package_id)
REFERENCES packages(package_id)) ENGINE=InnoDB;
```

```
CREATE TABLE grouptripparticipants (traveller_id INT, group_trip_id INT, package_id INT, PRIMARY KEY
(traveller_id, group_trip_id, package_id), CONSTRAINT fk_gtp_trav FOREIGN KEY (traveller_id) REFERENCES
travellers(user_id), CONSTRAINT fk_gtp_gt FOREIGN KEY (group_trip_id, package_id) REFERENCES
grouptrips(group_trip_id, package_id)) ENGINE=InnoDB;
```

```
CREATE TABLE groupmessages (message_id INT AUTO_INCREMENT PRIMARY KEY, group_trip_id INT, package_id INT, traveller_id INT, message TEXT, CONSTRAINT fk_gm_gt FOREIGN KEY (group_trip_id, package_id) REFERENCES grouptrips(group_trip_id, package_id), CONSTRAINT fk_gm_trav FOREIGN KEY (traveller_id) REFERENCES travellers(user_id)) ENGINE=InnoDB;
```

The following visual relational schema depicts the normalized database architecture for the Tripistry platform, illustrating primary and foreign key constraints as implemented in MariaDB.



Task 11:

Enhanced UI/UX (Task 11.1)

Several interface enhancements were implemented. An animated loading screen with sequenced step indicators and connecting line animations plays during booking processing, providing visual feedback for the multi-step background operation. The landing page features two carousels — Top-Rated Packages and Most Popular Packages — populated from live database queries. A light/dark theme toggle persists the user's preference via localStorage, with a pre-CSS script that applies the stored theme before the page renders to eliminate any flash of unstyled content. A full design system (StyleGuide) was produced documenting glassmorphism card variants, ambient background orbs, skeleton loaders, and micro-animations used consistently across the application.

Travellers can save packages to a personal favourites list directly from the packages listing page. The system uses a polymorphic favouritable table design that supports favouriting both packages and agencies under a single unified structure. The heart toggle on each package card reflects the current favourite state, pre-loaded per user on page render. Saved packages are accessible from the traveller dashboard with quick links to view or remove them.

Security Features (Task 11.2):

Multiple security improvements were implemented. Login rate-limiting locks an account for 5 minutes after 5 consecutive failed attempts. Session fixation is prevented by calling `session_regenerate_id(true)` on every successful login. Full session destruction is performed on logout, including cookie invalidation. Passwords are hashed using PHP's `PASSWORD_DEFAULT` bcrypt algorithm. Input sanitisation using `htmlspecialchars` and `strip_tags` is applied throughout all form submissions, and all database queries use PDO prepared statements to prevent SQL injection.

Algorithmic Thinking (Task 11.3)

When a traveller books a package, a matching algorithm in `BookingModel` automatically assigns them to an existing open group trip departing on the same date if one has available capacity. A `SELECT ... FOR UPDATE` lock prevents race conditions when multiple bookings occur simultaneously. If no suitable group exists, a new one is automatically created and the traveller becomes its first member. The traveller is then enrolled in the `grouptripparticipants` roster. This removes the need for travellers to manually find and join groups.

AI Integration (Task 11.4):

When a traveller completes a booking, the system calls the Google Gemini API with a structured prompt requesting exactly 3 destination-specific packing items and 2 local cultural etiquette tips. The response is parsed and stored as JSON in a `prep_notes` column on the booking record. This "Mission Intel" is then rendered directly on the traveller dashboard under each upcoming trip, giving travellers personalised, AI-generated travel preparation guidance tied to their specific destination.

Task 7:

The query joins four tables — packages, agencies, packagereviews, and favourites — filtering by status, grouping by package_id, and sorting by base_price. The EXPLAIN output before optimisation shows that the packagereviews table (r) is accessed with type ALL and key NULL, meaning MariaDB performed a full table scan with no index, examining all 6 rows on every join iteration. The packages table (p) similarly shows type ALL across 11 rows despite idx_packages_status_price existing as a possible key — the optimiser chose not to use it.

The optimisation applied was creating a dedicated index on packagereviews.package_id, since this column is the join condition between packages and packagereviews and would be evaluated on every row in a production dataset.

The EXPLAIN output after the index shows that idx_package_reviews_package_id now appears in the possible_keys column for the packagereviews row, confirming MariaDB is aware of it. However the key column remains NULL, meaning the optimiser still chose a full table scan. This is expected behaviour with a small dataset — MariaDB's cost-based optimiser determined that scanning 6 rows directly is cheaper than the overhead of index lookups. The spec explicitly warns of this: "Ensure that you have sufficient data in your database, or else you might not observe a meaningful difference."

Despite the EXPLAIN output appearing identical, the real execution time recorded in the database client dropped from 37ms to 4ms between the first and second runs, representing approximately a 9x improvement

```
home > shak > .dbclient > storage > 1779379940822@127.0.0.1@3306@tripsty > Tripsty.sql > ...
> Run | SQL
1 EXPLAIN SELECT p.package_id, p.title, p.base_price, p.duration_days,
2               p.cover_image_url, p.favouritable_id,
3               a.agency_name,
4               COALESCE(AVG(r.rating), 0) AS avg_rating,
5               COUNT(DISTINCT r.review_id) AS review_count
6 FROM packages p
7 LEFT JOIN agencies a ON p.agency_id = a.user_id
8 LEFT JOIN packagereviews r ON p.package_id = r.package_id
9 LEFT JOIN favourites f ON p.favouritable_id = f.favouritable_id AND f.traveller_id = 1
10 WHERE p.status = 'active'
11 GROUP BY p.package_id
12 ORDER BY p.base_price ASC
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
> 1	SIMPLE	p	ALL	idx_packages_status_price	(NULL)	(NULL)	(NULL)	11	Using where; Using temporary
> 1	SIMPLE	a	eq_ref	PRIMARY	PRIMARY	4	Tripsty.p.agency_id	1	
> 1	SIMPLE	r	ALL	idx_package_reviews_package_id	(NULL)	(NULL)	(NULL)	6	Using where; Using join buffer

```
> Run
CREATE INDEX idx_packagereviews_package_id ON packagereviews(package_id); 25ms
```

```
home > shak > .dbclient > storage > 1779379940822@127.0.0.1@3306@tripsty > Tripsty.sql > ...
2026-05-25 23:39:40 [DEBUG] Creating MariaDB connection for 1779379940822
2026-05-25 23:39:40 [INFO] Executing: EXPLAIN SELECT p.package_id, p.title, p.base_price, p.duration_days, p.cover_image_url, p.favouritable_id, a.agency_name, COALESCE(AVG(r.rating), 0) AS avg_rating, COUNT(DISTINCT r.review_id) AS review_count FROM packages p LEFT JOIN agencies a ON p.agency_id = a.user_id LEFT JOIN packagereviews r ON p.package_id = r.package_id LEFT JOIN favourites f ON p.favouritable_id = f.favouritable_id AND f.traveller_id = 1 WHERE p.status = 'active' GROUP BY p.package_id ORDER BY p.base_price ASC LIMIT 12 OFFSET 0
2026-05-25 23:39:40 [INFO] Result: 3 rows retrieved in 37ms
2026-05-25 23:40:12 [INFO] Executing: CREATE INDEX idx_packagereviews_package_id ON packagereviews(package_id)
2026-05-25 23:40:12 [INFO] Result: 0 rows affected in 66ms
2026-05-25 23:40:12 [DEBUG] Creating MariaDB connection for 1779379940822_tree
2026-05-25 23:40:31 [INFO] Executing: EXPLAIN SELECT p.package_id, p.title, p.base_price, p.duration_days, p.cover_image_url, p.favouritable_id, a.agency_name, COALESCE(AVG(r.rating), 0) AS avg_rating, COUNT(DISTINCT r.review_id) AS review_count FROM packages p LEFT JOIN agencies a ON p.agency_id = a.user_id LEFT JOIN packagereviews r ON p.package_id = r.package_id LEFT JOIN favourites f ON p.favouritable_id = f.favouritable_id AND f.traveller_id = 1 WHERE p.status = 'active' GROUP BY p.package_id ORDER BY p.base_price ASC LIMIT 12 OFFSET 0
2026-05-25 23:40:31 [INFO] Result: 3 rows retrieved in 4ms
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
> 1	SIMPLE	p	ALL	idx_packages_status_price	(NULL)	(NULL)	(NULL)	11	Using where; Using temporary
> 1	SIMPLE	a	eq_ref	PRIMARY	PRIMARY	4	Tripsty.p.agency_id	1	
> 1	SIMPLE	r	ALL	idx_package_reviews_package_id	(NULL)	(NULL)	(NULL)	6	Using where; Using join buffer

